Pr Brahim BAKKAS

ENSAM-Meknès
Génie Industriel - Intelligence Artificielle et Data Science

14 avril 2026

Installer et configurer un projet Django de gestion de stock.

Comprendre l’architecture **MVT** appliquée à un cas réel.

Construire les modèles Category et Product.

Gérer les **photos**, **détails**, **prix**, **stock** et **catégories**.

Mettre en place des **rôles utilisateurs** avec restrictions d’accès.

Réaliser une interface métier avec CRUD, recherche, tri, filtres et pagination.

1 Positionnement du TP
2 Modélisation UML et architecture MVT
3 Mise en place du projet
4 Modélisation des données
5 Administration Django
6 Gestion des rôles et permissions
7 Formulaires Django
8 Vues métiers
9 Routage des URL
10 Templates et interface utilisateur
11 Authentification et

groupes

12 Améliorations et bonnes pratiques

Pr Brahim BAKKAS

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

La gestion de stock est une application classique en entreprise.
Elle mobilise les notions importantes de Django :

- modèles,
- formulaires,
- vues,
- templates,
- permissions,
- upload d’images.

Elle constitue une excellente base pour aller vers :

gestion commerciale,
commandes,
fournisseurs,
ventes,
tableaux de bord.

Ce TP n’est pas un simple exercice technique : il simule une vraie application métier.

Pr Brahim BAKKAS

Élément Description

Projet gestion_stock
Application principale inventory

Fonction métier gestion des catégories et des produits **Données produit** nom, description, prix, stock, photo, catégorie, statut **Profils** superadmin, admin, utilisateur simple **Fonctions** CRUD, recherche, filtres, tri, pagination, visualisation **Objectif pédagogique** maîtriser une application Django complète avec rôles

Profil Droits

Superadmin CRUD catégories, CRUD produits, accès complet au back-office
Admin CRUD produits uniquement
Utilisateur simple consultation, détail, recherche, tri, filtrage

Implémentation conseillée :

utiliser **User** de Django,

créer des **Groupes** : superadmin, admin, viewer,

contrôler l’accès par décorateurs et tests de groupes.

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

6 Gestion des rôles et permissions

7 Formulaires Django

8 Vues métiers

Pr Brahim BAKKAS

La modélisation permet de transformer un besoin fonctionnel en une structure

claire. Elle aide à identifier :

- les acteurs ;
- les objets métiers ;
- les relations ;
- les flux d'information ;
- les responsabilités techniques.

Elle réduit les erreurs de conception avant l'écriture du code.

Elle facilite la communication entre enseignants, apprenants et développeurs.

Dans ce projet, la modélisation doit être comprise à trois niveaux :

fonctionnel : qui fait quoi ?

structurel : quelles données sont manipulées ?

technique : comment Django organise l'application ?

Élément Description

Superadmin gère les catégories, les produits et le paramétrage global
Admin gère les produits uniquement
Utilisateur simple consulte les produits, recherche, filtre et trie
Category organise les produits par famille ou type
Product contient les informations métier de l'article
Application Django traite les requêtes, applique les règles et affiche les résultats

Le noyau du projet repose sur deux entités principales : Category et Product, avec des droits différents selon le profil utilisateur.

Le diagramme de cas d’utilisation décrit les **services rendus** par le système. Il répond à la question :

Quelles actions chaque acteur peut-il réaliser ?

Il est particulièrement utile pour distinguer les droits :

- du superadmin,
- de l’admin,
- de l’utilisateur simple.

Le diagramme de cas d’utilisation met l’accent sur les **fonctionnalités**, pas encore sur les tables ou le code.

Pr Brahim BAKKAS

Superadmin

Se connecter

Consulter les produits

Voir le détail d’un produit

Rechercher / filtrer / trier

CRUD produits

CRUD catégories

simple

Pr Brahim BAKKAS

Admin

Utilisateur

Les trois profils peuvent se connecter et consulter les produits.

L’utilisateur simple reste limité à la **visualisation**.

L’admin peut gérer les **produits**, mais pas les catégories.

Le superadmin dispose du **contrôle total**.

Ce diagramme sert de base pour définir les permissions et pour construire les vues protégées dans Django.

Le diagramme de flux décrit l'**enchaînement des étapes** dans le système. Il montre :

- les entrées,
- les décisions,
- les traitements,

les sorties.

Il est utile pour comprendre le comportement global de l’application.

Pr Brahim BAKKAS

Début

Authentification

Profil ?

super

Admin

user

admin

Contrôle total

Utilisateur

Superadmin

Affichage

CRUD produits

Accès BDD

Consultation

Pr Brahim BAKKAS

Toute interaction commence par l’identification de l’utilisateur.

Le système applique ensuite une décision basée sur le rôle.

Les traitements autorisés conduisent à une lecture ou écriture en base.

Le résultat est finalement affiché sous forme HTML.

Le diagramme de flux prépare naturellement la compréhension des vues Django et du routage des requêtes.

Le diagramme de classes décrit la **structure statique** du système. Il présente :

les classes ;
leurs attributs ;
les relations entre elles.

Dans ce projet, les classes principales sont :

User,
Category,
Product.

gère
User

~~+ username : CharField~~

+ name : CharField
+ description : TextField + created_at : DateTimeField

Category

~~+ id : int~~

contient 1

~~+ id : int~~
+ name : CharField
+ description : TextField
+ price : DecimalField
+ stock : PositiveIntegerField
+ photo : ImageField
+ created_at : DateTimeField
+ updated_at : DateTimeField
+ category : ForeignKey(Category)

Pr Brahim BAKKAS
+ email : EmailField
+ password : CharField
+ groups : ManyToManyField

gère et consulte

Product

Une catégorie peut contenir plusieurs produits.
Chaque produit appartient à une seule catégorie.
Les utilisateurs interagissent avec les produits selon leur rôle.
Les catégories sont gérées uniquement par le superadmin.

Dans Django, la relation principale sera modélisée par :

```
category = models.ForeignKey(Category, on_delete=models.CASCADE)
```

Concept UML Traduction dans Django

Classe modèle dans models.py
Attribut champ Django (CharField, DecimalField, etc.)
Association 1..N ForeignKey
Acteur utilisateur authentifié
Rôle groupe Django ou permission
Action vue liée à une URL

Le diagramme de classes constitue le pont entre la conception UML et le code réel de models.py.

Le diagramme de séquence décrit le **déroulement temporel** d'une opération. Il répond à la question :

Dans quel ordre les objets interagissent-ils ?

Il est particulièrement utile pour expliquer une opération

comme : ajouter un produit ;

modifier un produit ;

consulter la liste des produits.

View
product_create

Product

Utilisateur Template

Formulaire
saisie du formulaire

validation et sauvegarde

objet enregistré
Model

POST + fichiers

message + redirection

page mise à jour

Pr Brahim BAKKAS

L’utilisateur remplit le formulaire.

Le template envoie les données à la vue.

La vue valide les données et appelle le modèle.

Le modèle enregistre le produit dans la base.

La vue renvoie un message et une redirection.

L’utilisateur obtient la nouvelle page.

Le diagramme de séquence complète le diagramme de classes : l’un montre la structure, l’autre montre le déroulement.

Django repose sur l’architecture **MVT** :

Model : structure des données ;
View : logique de traitement ;
Template : interface HTML.

Cette séparation améliore :

la lisibilité ;
la maintenance ;
la réutilisabilité ;
l'évolution future du projet.

MVT joue un rôle très proche de MVC, avec une organisation propre à Django.

requête HTTP

Utilisateur View

views.py

réponse HTML
Model
models.py

données lecture / écriture

Template
.html

contexte

Pr Brahim BAKKAS

Composant MVT Exemple concret

Model Product et Category
View product_list(request)
Template inventory/product_list.html
URL path(”, views.product_list, name=’product_list’)

Scénario :

l’utilisateur demande la page produit ;

la vue récupère les données depuis le modèle ;

la vue prépare le contexte ;

le template affiche la liste.

Composant MVT Rôle dans l’ajout

Model définit les champs du produit
View reçoit le formulaire, valide et enregistre
Template affiche le formulaire HTML
URL relie l’adresse /products/add/ à la vue

Grâce à MVT, chaque partie du projet a une mission claire, ce qui évite de mélanger données, logique et affichage.

Pr Brahim BAKKAS

Niveau Élément du projet

Cas d’utilisation fonctionnalités offertes à chaque profil

Diagramme de flux circulation globale des opérations
Diagramme de classes modèles Category et Product
Diagramme de séquence déroulement d'une opération métier **Architecture MVT** organisation technique Django

Pr Brahim BAKKAS

Le **diagramme de cas d'utilisation** décrit les fonctionnalités selon les

acteurs. Le **diagramme de flux** montre l'enchaînement global des traitements. Le **diagramme de classes** structure les données du système. Le **diagramme de séquence** explique le déroulement d'une opération. L'**architecture MVT** relie cette modélisation à l'implémentation Django.

Cette modélisation constitue une base solide avant de passer au code.

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

6 Gestion des rôles et permissions 7 Formulaires Django

8 Vues métiers

Pr Brahim BAKKAS

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# Linux / macOS
source .venv/bin/activate

pip install django Pillow
pip freeze > requirements.txt
```

Django : framework web Python.

Pillow : indispensable pour gérer les images.

requirements.txt : utile pour reproduire le projet.

```
django-admin startproject gestion_stock .
python manage.py startapp inventory
```

manage.py

gestion_stock/settings.py inventory/models.py

inventory/forms.py

inventory/views.py

inventory/urls.py

templates/inventory/

media/

Arborescence minimale :

Pr Brahim BAKKAS

```
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
```

```
        'django.contrib.sessions',
        'django.contrib.messages',
        'django.contrib.staticfiles',
        'inventory', #placer ici le nom de lapplication ]
LANGUAGE_CODE = 'fr-fr'
TIME_ZONE = 'Africa/Casablanca'
STATIC_URL = 'static/'
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

oublier inventory dans INSTALLED_APPS,et aussi oublier MEDIA_ROOT, confondre static et media.

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

6 Gestion des rôles et permissions

7 Formulaires Django

8 Vues métiers

Pr Brahim BAKKAS

```python
from django.db import models
class Category(models.Model):
    name = models.CharField("Nom", max_length=120, unique=True)
    description = models.TextField("Description", blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    class Meta:
        ordering = ['name']
        verbose_name = "Catégorie"
        verbose_name_plural = "Catégories"
    def __str__(self):
        return self.name
```

Une catégorie permet d'organiser les produits : informatique, bureautique, alimentation, etc.

```
class Product(models.Model):
    STATUS_CHOICES = [ ('available', 'Disponible'), ('low', 'Stock faible'),('out', 'Rupture')] name =
    models.CharField("Nom du produit", max_length=150)
    description = models.TextField("Description", blank=True)
    price = models.DecimalField("Prix", max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField("Quantité en stock", default=0)
    photo = models.ImageField(upload_to='products/', blank=True, null=True)
    category = models.ForeignKey(
        Category, on_delete=models.CASCADE, related_name='products' )
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True) class
    Meta:
        ordering = ['name']
        verbose_name = "Produit"
        verbose_name_plural ="Produits"
    def __str__(self):
        return self.name
    @property
    def stock_status(self):
        if self.stock == 0:
            return 'out'
        elif self.stock <= 5:
            return 'low'
        return 'available'
```

Champ Rôle

name nom de l’article
description détail ou fiche descriptive
price prix unitaire
stock quantité disponible
photo image du produit
category catégorie associée
created_at date de création
updated_at dernière modification

La propriété stock_status évite de stocker un état redondant dans la base.

```
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
```

Tester ensuite l’interface admin de Django pour vérifier que les modèles ont bien été créés.

Pr Brahim BAKKAS

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

6 Gestion des rôles et permissions 7 Formulaires Django

8 Vues métiers

Pr Brahim BAKKAS

```
from django.contrib import admin
from .models import Category, Product
@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ('name', 'created_at')
    search_fields = ('name',)
```

```
@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = (
        'name', 'category', 'price',
        'stock', 'created_at' )
    search_fields = ('name', 'description')
    list_filter = ('category', 'created_at')
```

L'admin Django sert ici de laboratoire de test avant de créer l'interface utilisateur finale.

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

6 Gestion des rôles et permissions

7 Formulaires Django

8 Vues métiers

Pr Brahim BAKKAS

On garde le modèle User par défaut de Django.

On crée trois groupes :

- superadmin
- admin
- viewer

Les droits sont définis :

- par les groupes,
- par des décorateurs,
- par des vérifications dans les vues.

Pour un TP initial, cette approche est plus simple qu’un modèle utilisateur personnalisé.

```
def is_superadmin(user):
    return user.is_authenticated and (
        user.is_superuser or
        user.groups.filter(name='superadmin').exists() )

def is_admin(user):
    return user.is_authenticated and (
        user.groups.filter(name='admin').exists() )

def is_viewer(user):
    return user.is_authenticated and (
        user.groups.filter(name='viewer').exists() )
```

On utilisera ces fonctions avec user_passes_test pour limiter l'accès à certaines vues.

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

6 Gestion des rôles et permissions

7 Formulaires Django

```
from django import forms
from .models import Category, Product

class CategoryForm(forms.ModelForm):
    class Meta:
        model = Category
        fields = ['name', 'description']
        widgets = {
            'name': forms.TextInput(attrs={'class': 'form-control'}), 'description':
            forms.Textarea(attrs={
                                    'class': 'form-control',
        }),                         'rows': 3
        }
```

```
class ProductForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = [
            'name', 'description', 'price',
            'stock', 'photo', 'category' ]
        widgets = {
            'name': forms.TextInput(attrs={'class': 'form-control'}), 'description':
            forms.Textarea(attrs={
                                    'rows': 4
            }),
            'class': 'form-control',

            'price': forms.NumberInput(attrs={
                                    'class': 'form-control',
            }),                     'step': '0.01'
```

```
        'stock': forms.NumberInput(attrs={'class': 'form-control'}), 'category':
forms.Select(attrs={'class': 'form-select'}), }
```

ModelForm simplifie la validation, la saisie et la maintenance.

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

6 Gestion des rôles et permissions 7 Formulaires Django

8 Vues métiers

Pr Brahim BAKKAS

```python
from django.shortcuts import render, redirect, get_object_or_404, from django.contrib.auth.dec
from django.core.paginator import Paginator
from django.db.models import Q
from .models import Product, Category

@login_required
def product_list(request) :
    query = request.GET.get('q', '')
    category_id = request.GET.get('category', '')
    sort = request.GET.get('sort', 'name')
    products = Product.objects.select_related('category').all()
    if query :
        products = products.filter(
            Q(name__icontains=query) |
            Q(description__icontains=query)
        )
    if category_id :
        products = products.filter(category_id=category_id)
    if sort in ['name', '-name', 'price', '-price'] :
        products = products.order_by(sort)
    paginator = Paginator(products, 6)
    page_obj = paginator.get_page(request.GET.get('page'))
    categories = Category.objects.all()
    return render(request, 'inventory/product_list.html', {
        'page_obj' : page_obj, 'categories' : categories,
        'query' : query, 'category_id' : category_id, 'sort' : sort,
    })
```

Points clés :

Filtrage : Utilisation de Q pour les recherches textuelles complexes.

Optimisation :
select_related réduit le nombre de requêtes SQL.

Pagination : Découpage des résultats en pages de 6
articles.

Tri : Gestion dynamique via les paramètres URL.

```
@login_required
def product_detail(request, pk):
    product = get_object_or_404(Product, pk=pk)
    return render(request, 'inventory/product_detail.html', {
        'product': product
    })
```

L'utilisateur simple doit au minimum pouvoir :

consulter la liste,

filtrer,

trier,

voir le détail d'un produit.

```python
def can_manage_products(user) :
    return user.is_authenticated and(
        user.is_superuser or
        user.groups.filter(name='superadmin').exists() or
        user.groups.filter(name='admin').exists()
    )
@login_required
@user_passes_test(can_manage_products)
def product_create(request) :
    form = ProductForm(request.POST or None, request.FILES or None)
    if form.is_valid() :
        form.save()
        messages.success(request,"Produit ajouté avec succès.")
        return redirect('product_list')
    return render(request, 'inventory/product_form.html', {'form' : form})
@login_required
@user_passes_test(can_manage_products)
def product_update(request, pk) :
    product = get_object_or_404(Product, pk=pk)
    form = ProductForm(request.POST or None, request.FILES or None, instance=product)
    if form.is_valid() :
        form.save()
        messages.success(request,"Produit modifié avec succès.")
        return redirect('product_list')
    return render(request, 'inventory/product_form.html', {'form' : form})
@login_required
@user_passes_test(can_manage_products)
def product_delete(request, pk) :
    product = get_object_or_404(Product, pk=pk)
    if request.method == 'POST' :
        product.delete()
        messages.warning(request,"Produit supprimé.")
        return redirect('product_list')
    return render(request, 'inventory/product_confirm_delete.html', {'product' : product
    })
```

Pr Brahim BAKKAS

```
@login_required
@user_passes_test(is_superadmin)
def category_list(request) :
    categories = Category.objects.all()
    return render(request, 'inventory/category_list.html', { 'categories' : categories
    })
@login_required
@user_passes_test(is_superadmin)
def category_create(request) :
    form = CategoryForm(request.POST or None)
    if form.is_valid() :
        form.save()
        messages.success(request,"Catégorie ajoutée avec succès.")
        return redirect('category_list')
    return render(request, 'inventory/category_form.html', {'form' : form})
@login_required
@user_passes_test(is_superadmin)
def category_update(request, pk) :
    category = get_object_or_404(Category, pk=pk)
    form = CategoryForm(request.POST or None, instance=category)
    if form.is_valid() :
        form.save()
        messages.success(request,"Catégorie modifiée avec succès.")
        return redirect('category_list')
    return render(request, 'inventory/category_form.html', {'form' : form})
@login_required
@user_passes_test(is_superadmin)
def category_delete(request, pk) :
    category = get_object_or_404(Category, pk=pk)
    if request.method == 'POST' :
        category.delete()
        messages.warning(request,"Catégorie supprimée.")
        return redirect('category_list')
    return render(request, 'inventory/category_confirm_delete.html', { 'category' : category
```

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

6 Gestion des rôles et permissions

7 Formulaires

Vues métiers

Pr Brahim BAKKAS

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.product_list, name='product_list'),
    path('products/<int :pk>/', views.product_detail, name='product_detail'),

    path('products/add/', views.product_create, name='product_create'),
    path('products/<int :pk>/edit/', views.product_update, name='product_update'),
    path('products/<int :pk>/delete/', views.product_delete, name='product_delete'),

    path('categories/', views.category_list, name='category_list'),
    path('categories/add/', views.category_create, name='category_create'),
    path('categories/<int :pk>/edit/', views.category_update, name='category_update'),
    path('categories/<int :pk>/delete/', views.category_delete, name='category_delete'), ]
```

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('inventory.urls')), ]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL,
        document_root=settings.MEDIA_ROO
    T )
```

Pr Brahim BAKKAS

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

5 Gestion des rôles et permissions 6 Formulaires Django

8 Vues métiers

Pr Brahim BAKKAS

```
< !DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Gestion de stock</title>
  <link href="https ://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
</head>
<body class="bg-light">
<nav class="navbar navbar-expand-lg navbar-dark bg-primary">
  <div class="container">
    <a class="navbar-brand" href="{% url 'product_list' %}">
      Stock Produits
```

```
    </a>
  </div>
</nav>
<div class="container py-4">
  {% if messages %}
    {% for message in messages %}
      <div class="alert alert-{{ message.tags }}">{{ message }}</div> {%
    endfor %}
  {% endif %}
  {% block content %}{% endblock %}
</div>
</body>
</html>
```

Pr Brahim BAKKAS

```
{% extends 'inventory/base.html' %}
{% block content %}
<h1 class="mb-4">Liste des produits</h1>
<form method="get" class="row g-3 mb-4">
  <div class="col-md-4">
    <input type="text" name="q" value="{{ query }}"
           class="form-control" placeholder="Recherche...">
  </div>
  <div class="col-md-3">
    <select name="category" class="form-select">
      <option value="">Toutes les catégories</option>
      {% for cat in categories %}
        <option value="{{ cat.id }}">{{ cat.name }}</option>
      {% endfor %}
    </select>
  </div>
  <div class="col-md-3">
    <select name="sort" class="form-select">
      <option value="name">Nom A-Z</option>
      <option value="price">Prix croissant</option>
      </select>
  </div>
  <div class="col-md-2">
    <button class="btn btn-primary">Filtrer</button>
  </div>
</form>
```

```
<div class="row">
{% for product in page_obj %}
<div class="col-md-4 mb-4">
<div class="card h-100 shadow-sm">
{% if product.photo %}
<img src="{{ product.photo.url }}"
class="card-img-top" style="height :220px ;">
{% endif %}
<div class="card-body">
<h5>{{ product.name }}</h5>
<p>{{ product.category.name }}</p>
<p><strong>Prix :</strong> {{ product.price }} DH</p> <p><strong>Stock :</strong> {{
product.stock }}</p> <a href="{% url 'product_detail' product.id %}"
class="btn btn-outline-primary btn-sm">
Détail
</a>
</div>
</div>
</div>
{% endfor %}
</div>
{% endblock %}
```

```
{% extends 'inventory/base.html' %}
{% block content %}
<h2 class="mb-4">Formulaire produit</h2>

<form method="post" enctype="multipart/form-data" class="card card-body shadow-sm">
   {% csrf_token %}
   {{ form.as_p }}
   <button class="btn btn-success">Enregistrer</button>
</form>
{% endblock %}
```

L'interface visée doit être simple, lisible et professionnelle :

cartes Bootstrap,

image visible,

prix et stock bien affichés,

boutons d'action clairs.

```
{% extends 'inventory/base.html' %}
{% block content %}
<div class="card shadow-sm">
  <div class="row g-0">
    <div class="col-md-4">
      {% if product.photo %}
      <img src="{{ product.photo.url }}" class="img-fluid rounded-start"> {% endif %}
    </div>
    <div class="col-md-8">
      <div class="card-body">
        <h3>{{ product.name }}</h3>
        <p><strong>Catégorie :</strong> {{ product.category.name }}</p>
        <p><strong>Description :</strong> {{ product.description }}</p>
        <p><strong>Prix :</strong> {{ product.price }} DH</p>
        <p><strong>Stock :</strong> {{ product.stock }}</p>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

6 Gestion des rôles et permissions

7 Formulaires

Vues métiers

Pr Brahim BAKKAS

Se connecter à /admin

Créer les groupes :

- superadmin
- admin
- viewer

Créer des utilisateurs de test

Affecter chaque utilisateur à un groupe

Prévoir par exemple :

super1 : groupe superadmin

admin1 : groupe admin

user1 : groupe viewer

Pr Brahim BAKKAS

Le superadmin crée des catégories.

Le superadmin ajoute plusieurs produits avec photos. L'admin modifie le stock et le prix d'un produit. L'admin ne doit pas accéder au CRUD catégorie. L'utilisateur simple consulte les produits.

L'utilisateur simple recherche par nom.

L'utilisateur simple trie par prix ou stock.

Vérifier qu'un accès interdit est bien bloqué.

Pr Brahim BAKKAS

1 Positionnement du TP

2 Modélisation UML et architecture MVT

3 Mise en place du projet

4 Modélisation des données

5 Administration Django

Bonne pratique Pourquoi

Utiliser ModelForm validation plus propre **Séparer les rôles** sécurité et clarté métier **Utiliser select_related**

optimisation des requêtes **Prévoir une page détail** meilleure expérience utilisateur **Ajouter messages Django** retour visuel après action **Paginer la liste** meilleure lisibilité

alerte automatique de stock faible ;

tableau de bord statistique ;

export CSV ou Excel ;

gestion des fournisseurs ;

historique des mouvements de stock ;
recherche avancée multi-critères ;
authentification personnalisée.

Le TP peut évoluer progressivement vers une application de gestion commerciale complète.

Ajouter un champ reference au produit.
Ajouter un badge visuel pour stock faible ou rupture.

Interdire la suppression d’une catégorie contenant encore des produits. Ajouter une recherche par intervalle de prix.

Ajouter une page d’accueil avec indicateurs :

- nombre total de produits,
- nombre de catégories,
- produits en rupture.

Pr Brahim BAKKAS

Réaliser une version personnelle du système

avec : logo de l’établissement ;

authentification ;

gestion des catégories ;

gestion des produits avec photo ;

rôles utilisateurs ;

recherche, tri, filtrage ;

pagination ;

interface Bootstrap propre.